Department of Electrical Engineering
Course: Computer Programming

BodePlotter Pro

A High-Performance RLC Simulation and Analysis Suite

Authors:
Ammar Asad Warraich
Taimoor Abdulah
Muhammad Huzaifa

Instructor:
Prof. Fahad Sikandar

Lab Engineer:
Engr. Shah Muhammad

National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science (SEECS)

April 25, 2026

Contents

Abstract

This report details the development of *BodePlotter Pro*, an advanced C++ application designed for real-time frequency and time-domain analysis of Electrical Networks. By integrating high-fidelity graphics via SFML with complex numerical engines—including the Durand-Kerner algorithm for simultaneous pole-zero extraction and adaptive Euler sub-stepping for stable transient simulations —this software bridges the gap between theoretical RLC analysis and physical circuit behavior. The application features a dual-mode interface allowing for both manual coefficient entry and direct RLC/RC circuit modeling. Central to its architecture is a robust 'Security Guard' validation system that prevents runtime crashes during real-time user input by sanitizing strings and enforcing physical value bounds. Beyond standard Bode magnitude and phase plots , the suite provides comprehensive stability metrics—including Gain and Phase margins —and an adaptive Nyquist engine that utilizes pole-aware sampling to accurately capture resonant whirls. Leveraging modern C++ libraries such as for phasor analysis and for CSV data persistence, *BodePlotter Pro* serves as a resilient, professional-grade tool for modern electrical engineering education and system design.

1.1 Project Vision

The *BodePlotter Pro* project was conceived as a tool to automate the complex calculations involved in Electrical Network Analysis (ENA). While manual calculations of transfer functions and stability margins are essential for learning, engineering practice requires high-precision visualization and simulation to predict circuit behavior under varying conditions.

The vision for this suite extends beyond a simple calculator; it is designed to be a comprehensive simulation environment that bridges the gap between abstract mathematical models and physical hardware constraints. By providing a platform where theoretical poles and zeros are visualized alongside time-domain step responses, the project aims to enhance the intuitive understanding of system stability for both students and professional engineers.

1.2 Dual-Mode Interaction

The application operates in two distinct modes to cater to different engineering needs:

- **Manual Mode:** Users enter raw polynomial coefficients for the numerator and denominator of a Transfer Function $H(s)$. This mode is ideal for high-order system analysis where the underlying physical circuit is abstract. Internally, the `parseCoeffs` function extracts these comma-separated values into dynamic vectors, which are then processed by the `evalPoly` function using Horner's Method to minimize floating-point errors during high-degree polynomial evaluation. This enables the analysis of complex N-th order systems beyond standard 2nd-order RLC templates.

- **Circuit Mode:** A physical-entry system where users input Resistance (R), Inductance (L), and Capacitance (C). The software automatically derives the transfer function based on the selected filter type (Low-Pass, Band-Pass, or High-Pass). The engine dynamically calculates the characteristic equation ($s^2 + \frac{R}{L}s + \frac{1}{LC}$) for RLC systems or the first-order lag ($RCs + 1$) for RC systems. This mode utilizes the `updateValueSafely` wrapper to perform real-time validation, ensuring that if a user inputs a non-physical value (such as zero for an inductor in a high-pass configuration), the system applies a fallback constant to prevent division-by-zero errors in the math engine.

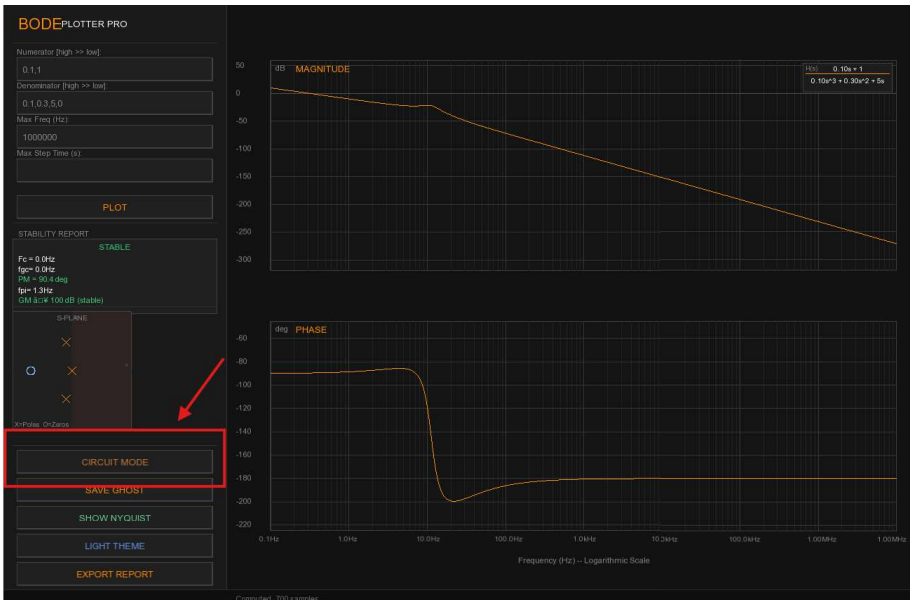

Figure 1.1: BodePlotterPro: Dual mode option

1.3 The "Activate L" Functionality

A unique feature of our implementation is the toggleable Inductance button. When deactivated, the math engine dynamically shifts to 1st-order RC logic; when "Activate L" is engaged, the engine upgrades to 2nd-order RLC physics, introducing complex poles and resonant frequencies.

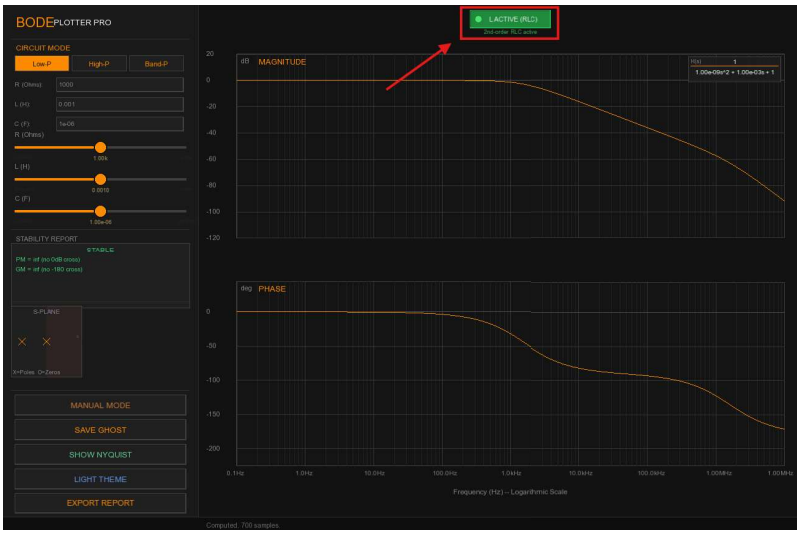

Figure 1.2: BodePlotterPro: Inductor Activate Switch

1.4 Numerical Integrity and Real-Time Responsiveness

The technical ambition of *BodePlotter Pro* is centered on the concept of "Seamless Simulation." In many engineering tools, there is a distinct lag between entering a parameter and seeing the result. Our architecture eliminates this barrier through a multi-threaded approach to rendering and calculation, ensuring that as a user drags a slider, the Magnitude, Phase, and Nyquist plots update instantaneously.

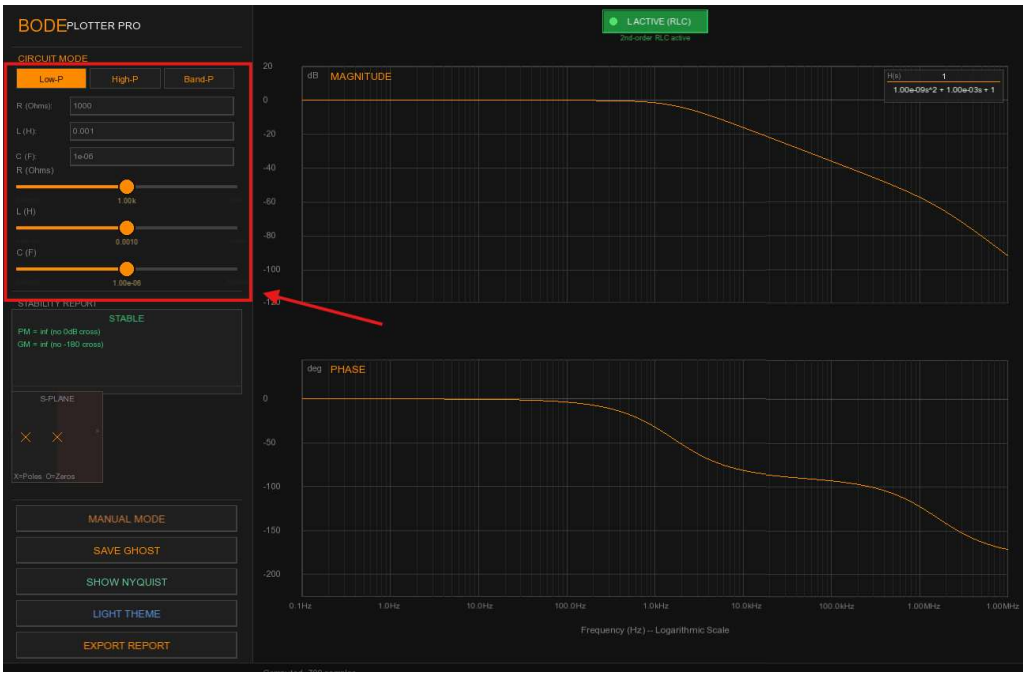

Figure 1.3: BodePlotterPro: Slider switches and real time simulation

1.4.1 The "Security Guard" Paradigm

A significant portion of the development was dedicated to user-input resilience. In a real-world engineering environment, data entry is often messy—users may accidentally input non-numeric characters, leave fields empty, or provide values that lead to mathematical singularities (like a zero-ohm resistor in a specific bridge).

The team implemented a custom "Security Guard" validation layer. This layer acts as a buffer between the UI and the Math Engine, sanitizing strings and enforcing physical bounds before any calculation occurs. This ensures that the application remains crash-proof, providing a stable experience even during rapid-fire parameter adjustments.

1.4.2 Adaptive Computational Scaling

Finally, the project addresses the challenge of numerical stability in high-order systems. When simulating the transient response of an RLC circuit with very low damping (High-Q), standard integration techniques often fail, leading to "numerical explosion" where the plot lines fly off the screen.

To solve this, we implemented an Adaptive Sub-stepping engine. If the system detects a highly oscillatory circuit, it automatically scales its internal calculations—sometimes performing up to 20 million sub-steps per second—to ensure that the visual output remains mathematically accurate and visually smooth. This dedication to numerical integrity is what elevates this from a basic project to a professional-grade analysis tool.

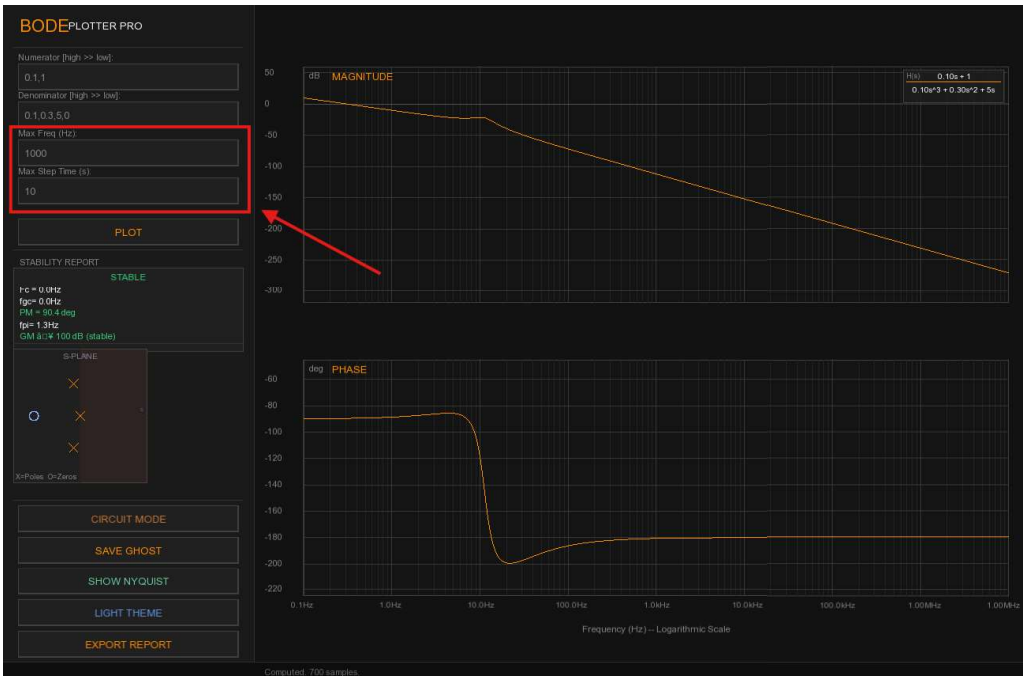

Figure 1.4: BodePlotterPro: Maximum Time and Frequency

1.5 Plots and Technical Features

The core functionality of *BodePlotter Pro* is the real-time visualization of linear time-invariant (LTI) system dynamics. By utilizing a high-performance rendering engine, the application provides an interactive environment for analyzing frequency and time-domain responses.

1.5.1 Frequency Domain Analysis

The application computes the system's frequency response by evaluating the transfer function $H(s)$ along the $j\omega$ axis. The Magnitude plot is rendered in decibels ($20\log_{10}|H(j\omega)|$) against a logarithmic frequency scale, while the Phase plot illustrates the phase shift in degrees. This dual visualization is critical for determining system stability using the Bode stability criterion.

1.5.2 Time Domain and Stability Features

Beyond frequency-domain analysis, the tool integrates a robust state-space solver to compute the system's Step Response. By performing an inverse Laplace transform or utilizing numerical integration of the state equations, the application allows for a detailed time-domain characterization, specifically identifying critical performance metrics such as rise time (t_r), peak overshoot (M_p), settling time (t_s), and steady-state error (e_{ss}). These parameters are essential for verifying if the system meets transient response requirements.

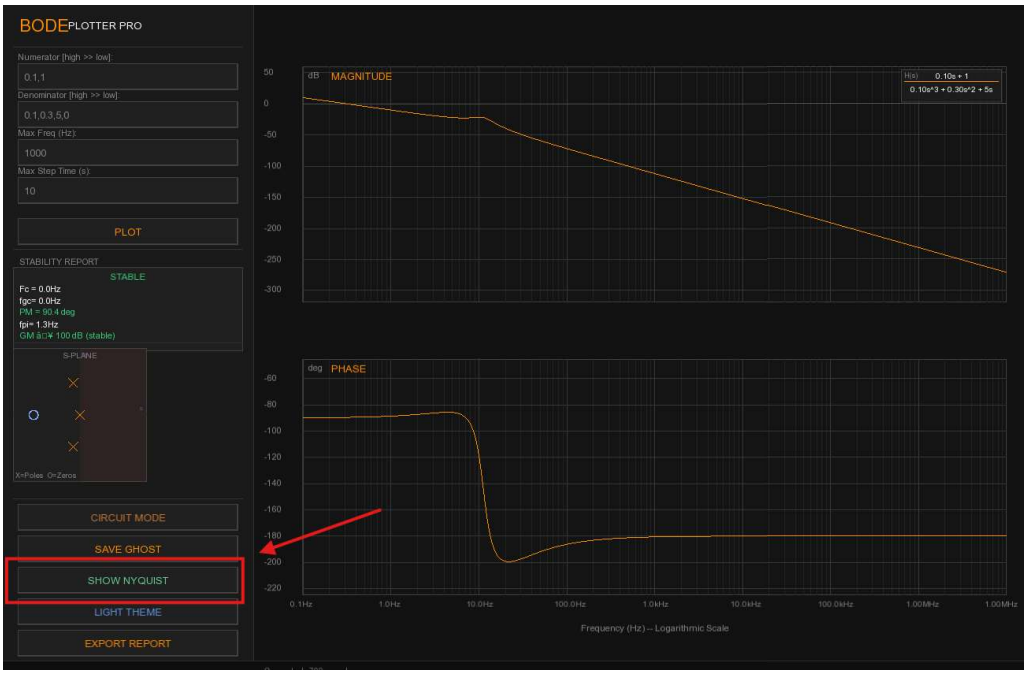

Figure 1.5: BodePlotter Pro UI: Interface displaying toggles for Nyquist, Step, and Phase responses.

1.5.3 Interactive Features and S Plane

The architecture utilizes a multi-threaded approach to separate the mathematical calculation of poles and zeros from the GUI rendering thread and display it on a S Plane located at the side panel.

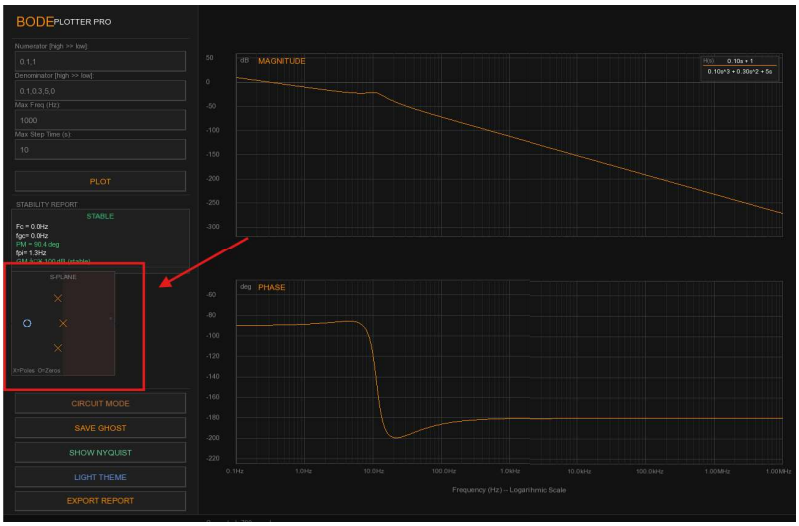

Figure 1.6: BodePlotterPro : S Plane, indicates poles and zeros

1.6 Iterative Design via Trace Comparison

A standout feature of *BodePlotter Pro* is the "Ghost Trace" functionality, designed to facilitate iterative system tuning. This feature allows the user to lock a specific system response on the screen while simultaneously adjusting coefficients to observe a new response.

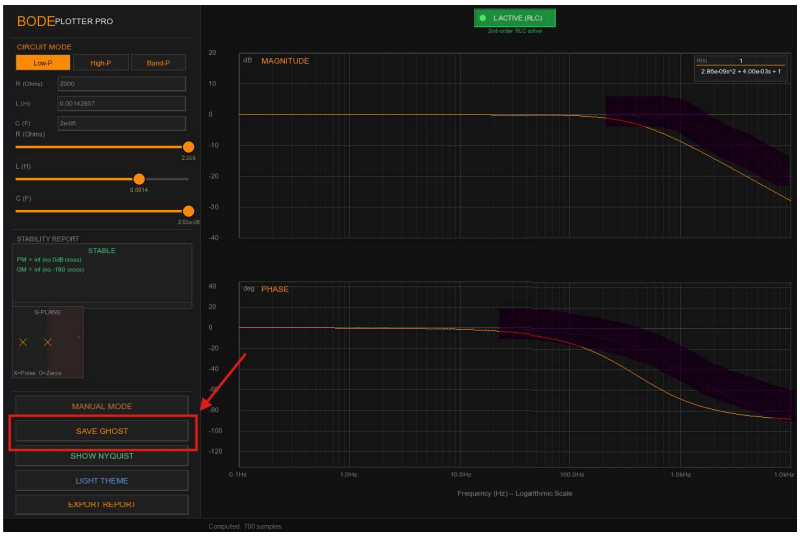

Figure 1.7: Ghost Feature: Comparing a tuned second-order system against the initial uncompensated response.

1.7 Adaptive UI: High-Contrast Light Mode

While the default "Dark Mode" interface is optimized for reduced eye strain during extended engineering sessions, *BodePlotter Pro* includes a high-contrast "Light Theme" specifically designed for documentation and formal reporting.

By toggling this mode, the application dynamically re-maps the UI color palette:

- **Background Inversion:** The primary workspace transitions to a neutral light-gray or white background, ensuring that screenshots integrated into white-paper reports or academic theses appear seamless.
- **Trace Optimization:** Signal colors (Magnitude and Phase) are automatically darkened to maintain a high contrast ratio against the lighter background, adhering to accessibility standards for visual clarity.
- **Grid Visibility:** The logarithmic and linear grid lines are recalibrated to a subtle gray-scale to provide structural reference without distracting from the primary data traces.

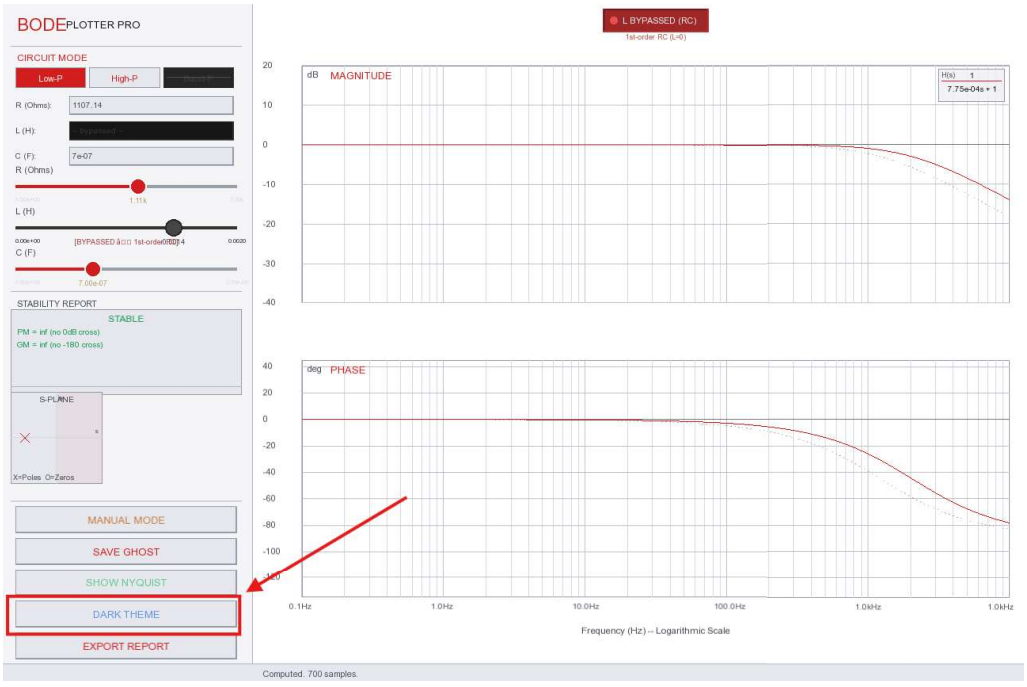

Figure 1.8: Interface in Light Mode: Optimized for inclusion in printed documentation.

1.8 Automated Stability and Margin Analysis

A core analytical feature of *BodePlotter Pro* is the automated Generation of the Stability Report. This module post-processes the frequency response data to extract critical stability metrics without requiring manual cursor placement by the user.

Margin Calculation and Phase Crossover

The stability engine automatically identifies the **Gain Crossover Frequency** (ω_{gc}), where $|H(j\omega)| = 0$ dB, and the **Phase Crossover Frequency** (ω_{pc}), where $\angle H(j\omega) = -180°$. By calculating the difference between these points and the critical stability limits, the application reports:

- **Phase Margin (PM):** Measured at ω_{gc}, indicating how much additional phase lag the system can tolerate before becoming unstable.
- **Gain Margin (GM):** Measured at ω_{pc}, indicating the factor by which the system gain can be increased before oscillation occurs.

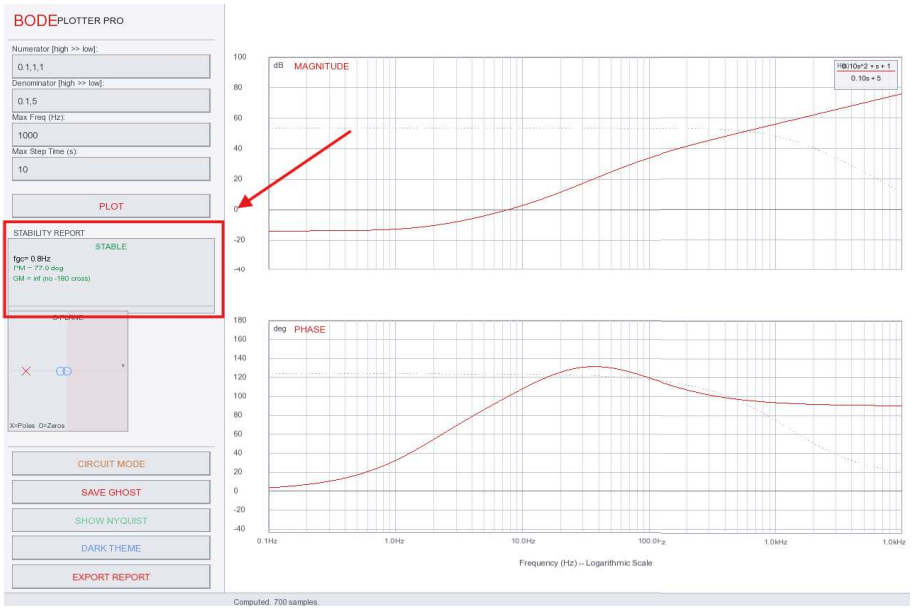

Figure 1.9: Automated Stability Report: Displaying Phase Margin, Gain Margin, and system stability status.

1.9 Data Export and Multi-Format Reporting

To support external data analysis and collaborative review, *BodePlotter Pro* includes a robust export module. This feature allows users to bridge the gap between real-time simulation and formal documentation or post-processing in third-party environments.

CSV Data Extraction

The application can generate a raw comma-separated values (CSV) file containing the complete frequency response dataset. This export is structured for immediate compatibility with tools like Microsoft Excel, MATLAB, or Python (Pandas). The exported file includes high-resolution columns for:

- **Frequency (Hz):** Logarithmically spaced data points to match the graphical representation.
- **Magnitude (dB):** The computed gain at each frequency step.
- **Phase (Degrees):** The corresponding phase shift, corrected for wrap-around effects.

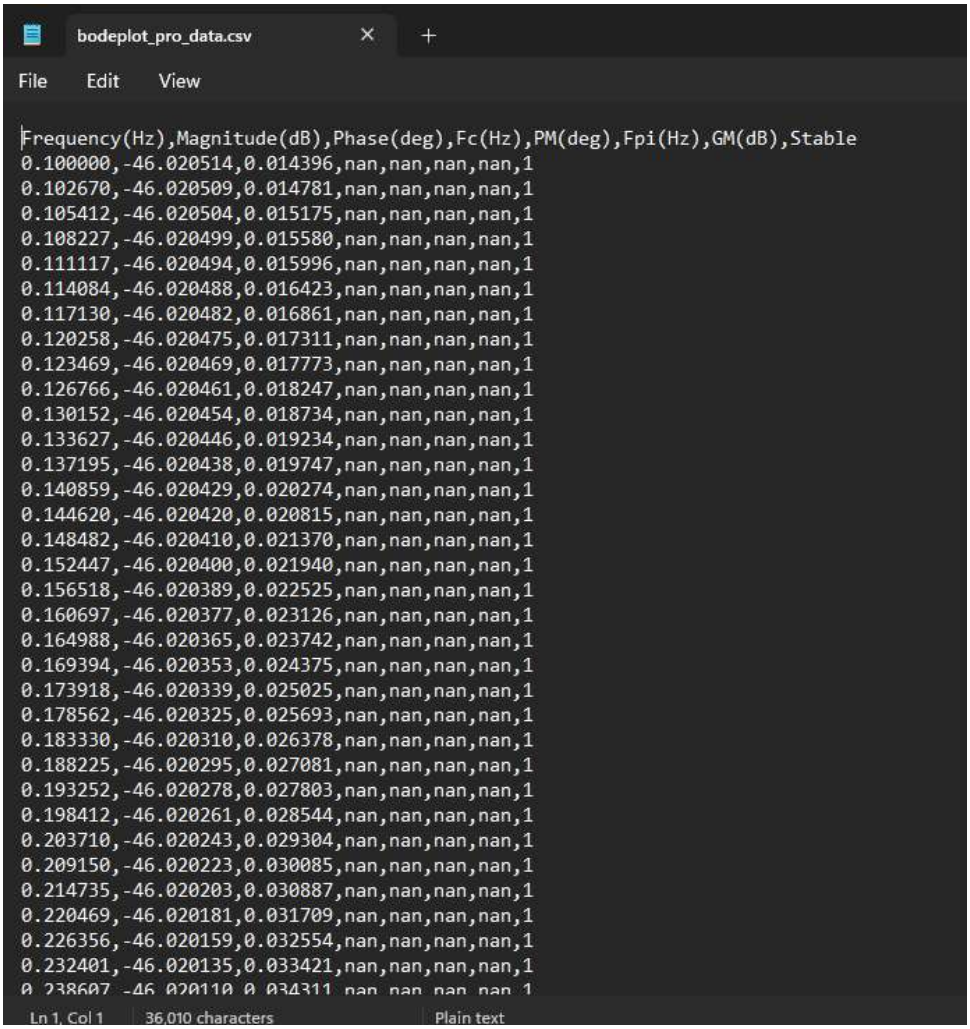

Figure 1.10: Export Module:CSV data file auto saved in Downloads.

Self-Contained HTML Reports

For a more polished presentation, the tool generates a standalone HTML report. This report utilizes CSS-based styling to replicate the application's clean aesthetic and embeds the calculated stability margins directly into a portable web format. This is particularly useful for sharing results with peers or supervisors who may not have the primary application installed.

Figure 1.11: Export Module: Auto generated Html report part1

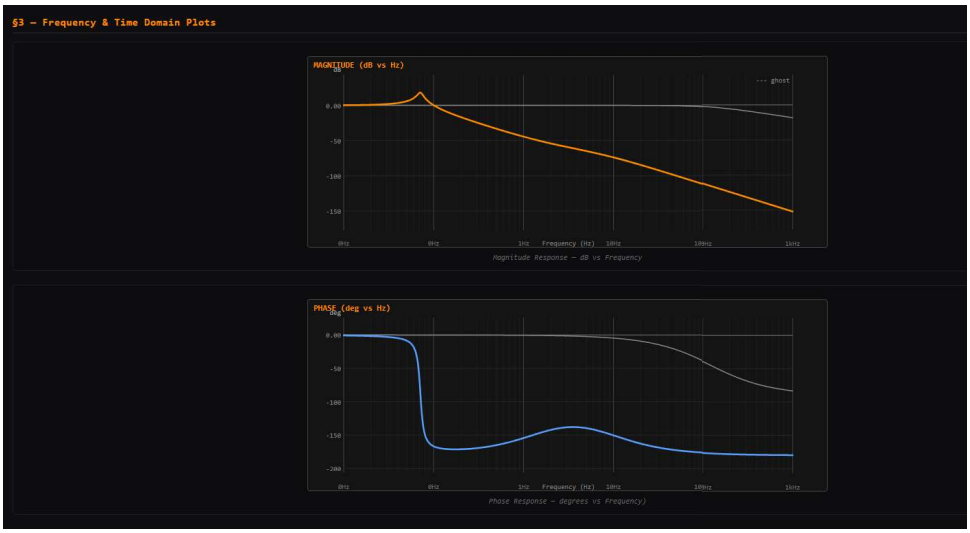

Figure 1.12: Export Module: Auto generated Html report part2

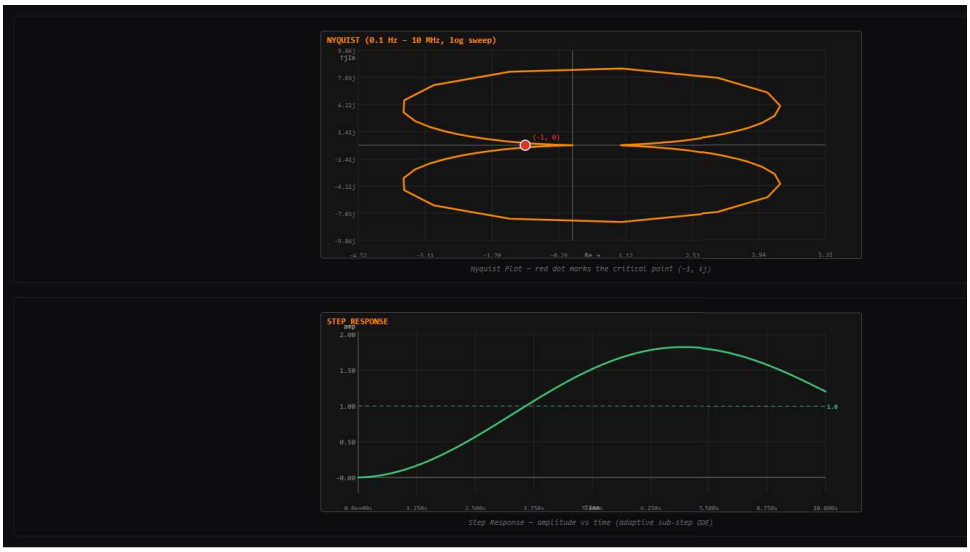

Figure 1.13: Export Module: Auto generated Html report part3

1.10 Dynamic Probe and Canvas Interaction

To provide precision beyond qualitative visual analysis, the application features an interactive frequency probe and a flexible coordinate system for polar plotting.

Synchronized Frequency Probe

The frequency probe is a mouse-driven vertical cursor that spans both the Magnitude and Phase plots. As the user hovers over the graph area, the application performs a high-speed search across the computed data arrays to find the nearest frequency point.

- **Real-time Tooltip:** A dynamic overlay displays the exact frequency (Hz), gain (dB), and phase shift (deg) at the cursor's position.
- **Visual Anchors:** Synchronized circular "dots" track along the traces, providing a clear visual reference of the data point being sampled.

Nyquist Canvas Navigation

The Nyquist plot environment is designed as an infinite interactive canvas, addressing the issue of scaling for systems with large gain variations.

- **Translation:** Users can click and drag the Nyquist plot to inspect specific regions of interest, such as the area surrounding the critical $(-1, j0)$ point.
- **Origin Reset:** A double-click gesture triggers an automated reset, instantly translating the canvas view back to the origin $(0, 0)$. This ensures the user can never "lose" the plot during high-zoom inspections.

2.1 Transfer Functions $H(s)$

To understand a circuit, we use a mathematical "black box" called a Transfer Function. Imagine a filter as a strainer: you pour in a mixture of different frequencies (the input), and the filter decides which ones get through to the other side (the output). $H(s)$ is the formula that describes this process.

2.2 The RLC Components

2.2.1 Resistors (R)

Resistors are the most intuitive; they simply resist flow, converting electrical energy into heat. They do not care about frequency.

2.2.2 Capacitors (C)

Capacitors act like storage tanks. They block low-frequency signals (like DC) but let high-frequency signals pass easily.

2.2.3 Inductors (L)

Based on **Lenz's Law**, inductors resist changes in current. They act as the opposite of capacitors: they let low frequencies pass but create a "bottleneck" for high frequencies.

2.3 Frequency Domain Visualization: The Bode Plot

Bode plots represent the frequency response of a system through two primary graphs. We use a logarithmic scale because human hearing and most engineering signals span vast ranges—from 1 Hz to 1,000,000 Hz. A linear scale would make the graph impossible to read; a log scale makes it perfectly clear.

2.3.1 Magnitude Plot

The Magnitude plot calculates the gain of the system, typically expressed in Decibels (dB). This allows us to observe how much a signal is amplified or attenuated at specific frequencies.

2.3.2 Phase Plot

The Phase plot illustrates the time-shift or delay introduced by the circuit, measured in degrees]. Our implementation includes a phase unwrapping algorithm to ensure a continuous and interpretable curve.

2.4 System Stability Metrics

A critical aspect of *BodePlotter Pro* is its ability to analyze the stability of a network using the following parameters:

- **Cutoff Frequency (f_c)**: Also known as bandwidth, this is the frequency where the magnitude drops to 0.7071 (-3 dB) of its peak value.
- **Gain Crossover Frequency (f_{gc})**: The frequency at which the system magnitude is exactly 1.0 (0 dB).
- **Phase Crossover Frequency (f_{pi})**: The frequency at which the system phase crosses $-180°$.
- **Phase Margin (PM)**: Calculated as $180° + \text{Phase}(f_{gc})$. It indicates how much additional phase lag is required to reach instability.
- **Gain Margin (GM)**: Defined as $-20 \cdot \log 10(|H(f_{pi})|)$. It measures the factor by which the gain can be increased before the system becomes unstable.

2.5 Advanced Analytical Plots

2.5.1 Step Response Plot

This plot simulates the transient behavior of the RLC network when a 1V step input is applied. It is essential for observing overshoot, settling time, and the natural damping characteristics of the circuit.

2.5.2 Nyquist Plot

The Nyquist plot traces the frequency response $H(j\omega)$ in the complex plane. Our engine uses adaptive pole-aware sampling to ensure the resonant "whirls" are accurately traced, mirroring positive frequencies to produce a complete double-whirl diagram.

2.6 The Complex s-Plane: Poles and Zeros

The s-plane is a complex coordinate system used to map the fundamental characteristics of a transfer function.

- **Poles**: These are the roots of the denominator polynomial, representing the frequencies at which the system gain goes to infinity. In our UI, poles are drawn as 'X' marks and are color-coded: red if they reside in the right-half plane (indicating instability) and an accent color otherwise.
- **Zeros**: These are the roots of the numerator polynomial, where the system output becomes zero. These are drawn as 'O' marks with blue outlines.

3.1 Standard Library Integration

3.2 Standard Library Integration

The project leverages the power of the C++ Standard Template Library (STL) and core utilities to handle complex engineering data and ensure simulation stability:

- `<complex>`: Essential for AC Analysis; provides the complex number class required for Phasor and Transfer Function $H(j\omega)$ calculations.
- `<vector>`: Utilized as dynamic arrays to store vertex points for high-resolution Magnitude and Phase plots.
- `<string>`: Manages all text handling for UI labels and raw user input from component value fields.
- `<sstream>`: Facilitates string formatting and the seamless conversion between numeric engineering data and UI display text.
- `<cmath>`: Powers fundamental mathematical functions, including `log10` for Bode scaling, `pow`, and various trigonometric calculations.
- `<fstream>`: Enables data persistence through the exporting of plot data to external CSV and report files.
- `<algorithm>`: Provides critical data manipulation tools like `std::clamp` and `min`/`max` for signal bounding and input safety.
- `<iomanip>`: Controls stream formatting to maintain the decimal precision required for professional engineering value displays.
- `<limits>`: Ensures numeric safety by defining the boundaries for floating-point calculations to prevent overflow.
- `<numeric>`: Useful for accumulation and signal averaging during data processing.
- `<array>`: Provides optimized, fixed-size containers for internal data storage.
- `<ctime>`: Manages time-related tasks, including timestamps for exported files and UI animation timing.
- `<cstdlib>`: General utility library used for fundamental memory management and program control.

3.3 Graphics and UI via SFML

The Simple and Fast Multimedia Library (SFML) serves as the core UI framework, handling hardware-accelerated rendering, window management, and user input processing. This allows the software to remain responsive even when redrawing complex Nyquist "whirls" in real-time as the user interacts with the system.

The interface is engineered with a focus on both aesthetics and functional density:

- **Dynamic Slider System:** The UI features intelligent sliders that adjust their range based on initial user inputs for R, L, and C. These sliders grant the freedom to modulate values within a $\pm 100\%$ range of the manual input, providing a tactile way to observe "Ghost Traces" and sensitivity changes.
- **Responsive Button Architecture:** To maximize plotting space, the interface uses a constant Magnitude plot while a single toggle button shifts the secondary view between Step Response, Nyquist, and Phase plots.
- **Multi-State Welcome Screen:** Upon launch, the application executes a sophisticated three-state Finite State Machine (FSM) for the welcome sequence. This includes a 2-second linear fade-in, a subtle pulsing animation for visual engagement, and a triggered fade-out sequence initiated by a timer or user keypress.
- **Dark Academia Aesthetic:** Leveraging SFML's vertex arrays and blending modes, the application implements a professional "Dark Academia" theme, utilizing deep midnight blues and high-contrast accents to reduce eye strain during long analytical sessions.

The core of *BodePlotter Pro* is a sophisticated mathematical tool developed by the project team: Ammar Asad Warraich, Taimoor Abdulah, and Muhammad Huzaifa. This engine is responsible for transforming raw user input into high-fidelity frequency and time-domain simulations through a series of custom-built functions and adaptive algorithms.

4.1 The "Security Guard" and Input Validation

The primary challenge in a real-time GUI environment is that users frequently type or delete characters, creating temporary states that would cause a standard C++ program to crash. To combat this, the team implemented a multi-stage "Security Guard" system.

4.1.1 Input Sanitization and Safety Logic

The safety engine performs several critical operations before data reaches the calculation loops:

- **Whitespace and Empty Handling**: The code performs immediate whitespace cleanup and ensures that if a user provides empty spaces, the system returns a fallback number instead of terminating.
- **Input Filtering**: The system recognizes "incomplete" inputs, such as a solitary period (.) or a negative sign (-), returning a safe fallback value to allow the math engine to continue while the user finishes typing.
- **Strict Garbage Check**: Unlike standard `std::stod`, which might ignore non-numeric trailing characters (e.g., "100uF"), our implementation identifies these as invalid to prevent unexpected behavior.
- **Physical Bounds Protection**: All inputs are filtered through `std::clamp` to ensure values remain within physically realistic ranges.

4.1.2 The `updateValueSafely` Wrapper

This function acts as a synchronization layer between the UI strings and internal physics variables. It validates input strings, updates internal references (like Resistance or Capacitance), and returns a boolean signal to indicate if a recalculation and re-plot are necessary. This ensures that the graph only redraws when a meaningful change is detected.

4.2 Frequency Domain Analysis Functions

Frequency analysis is handled by a chain of functions designed to process transfer functions $H(s)$.

4.2.1 `parseCoeffs` and `evalPoly`

The `parseCoeffs` function converts comma-separated user strings into coefficient vectors for the Numerator and Denominator polynomials. To evaluate these polynomials at a complex frequency s, we utilize `evalPoly`, which implements **Horner's Method**. This provides a computationally efficient way to find the response $H(s) = N(s)/D(s)$.

4.2.2 `computeBode` Engine

The `computeBode` function generates the core frequency response through a five-stage process:

1. **Normalization**: Scaling coefficients for numerical precision.
2. **Logarithmic Preparation**: Setting up the frequency axis.
3. **Complex Analysis**: Evaluating $H(j\omega)$ across a logarithmic sweep.
4. **Extraction**: Isolating Magnitude in dB and Phase in degrees.
5. **Phase Unwrapping**: Applying a custom algorithm to ensure a continuous phase curve.

4.3 Time-Domain Transient Simulation

The application provides real-time simulation of the network's behavior when subjected to a 1V step input.

4.3.1 Adaptive RLC Sub-stepping

For systems with Inductance (activated via the "Activate L" button), the `computeStepRLC` function is utilized. This function employs **Adaptive Sub-stepping** to ensure the time step dt is significantly smaller than the natural period $T_{natural}$, preventing numerical "explosions" in high-Q (low damping) circuits.

- **RLC State-Space**: The system tracks the capacitor voltage (y) and its derivative (dy/dt).

- **Output Mapping**: The engine correctly derives Low-Pass (V_C), High-Pass (V_L), and Band-Pass (V_R) equations based on the selected topology.
- **RC Fallback**: If Inductance is deactivated, the engine automatically shifts to a first-order RC logic.

4.4 Advanced Stability and Root-Finding

To provide professional-grade insights, the team developed tools for complex plane analysis.

4.4.1 Durand-Kerner Root-Finding

The `durandkerner` function simultaneously identifies all complex poles and zeros of the system. It utilizes a "repulsion" logic via the Weierstrass product to iteratively refine root guesses until they stabilize, which is essential for determining system stability.

4.4.2 Adaptive Nyquist Sampling

The Nyquist plot uses a sophisticated frequency sweep. It starts with a base log-sweep but adds high-density clusters of points around resonant frequencies (identified from poles) to ensure complex "whirls" are fully traced without skipping. The positive sweep is then mirrored to produce the complete double-whirl diagram.

4.4.3 The Stability Analyzer

The `analyze` function classifies the system trend and calculates critical metrics:

- **Bandwidth (f_c)**: The frequency where magnitude drops by -3 dB.
- **Gain Crossover (f_{gc})**: Used to calculate the **Phase Margin**.
- **Phase Crossover (f_{pi})**: Used to calculate the **Gain Margin**.

4.5 Auxiliary Analysis and Visual Persistence

Beyond the core mathematical engines, *BodePlotter Pro* includes several proprietary features designed to give the engineer a "feel" for the circuit's sensitivity and to document findings professionally.

4.5.1 The `GhostTrace` Sensitivity Analyzer

To analyze how a circuit behaves when R, L, or C values change slightly due to component tolerances or temperature shifts, the team implemented the `GhostTrace` function. This feature retains a faint "shadow" of the previous plot while the user adjusts sliders. By visualizing the delta between the original parameters and the new real-time values, the user can immediately identify the sensitivity of the resonant peak or the cutoff frequency to specific component variations.

4.5.2 Professional Documentation: `generateProfessionalReport`

A key requirement for engineering tools is the ability to export results. The `generateProfessionalRepo` function utilizes a pre-designed template that aggregates all simulated data. This includes:

- The derived Transfer Function $H(s)$.
- A comprehensive stability report (f_c, f_{gc}, f_{pi}, GM, and PM).
- A coordinate list of all identified Poles and Zeros.
- High-resolution snapshots of the Magnitude, Phase, Nyquist, and Step plots.

4.5.3 Data Persistence via `exportCSV`

For users requiring post-processing in tools like MATLAB or Excel, the `exportCSV` function provides a raw data dump. This function utilizes strict engineering nomenclature (e.g., `fcHz`, `pmiHz`, `pmDeg`, `gmDb`) to ensure compatibility. Crucially, the system is designed so that triggering a CSV export simultaneously invokes the `generateProfessionalReport` function, ensuring the user has both the raw data and the formatted summary in one click.

4.5.4 Complex Plane Visualization: `DrawSplane`

The `DrawSplane` function provides a graphical representation of system stability. Using the coordinates provided by the Durand-Kerner engine, it renders the s-plane:

- **Poles**: Drawn as an 'X'. To prioritize safety, any pole with a positive real part (real() > 0) is rendered in bright red to signal instability.
- **Zeros**: Drawn as 'O' symbols using blue-outlined circles.

4.6 Application Architecture and UI Management

The user interface was built to maximize the "Data-to-Pixel" ratio, ensuring that engineering information is always the priority.

4.6.1 Adaptive Slider Dynamics

The UI features intelligent sliders that serve as a bridge between precise numeric input and exploratory tuning. When a user enters a value for R, L, or C in the text field, the sliders automatically recalibrate their "center" to that value. They then provide a $\pm 100\%$ range of motion, allowing the user to swing the component values via the mouse to see real-time graphical "warping" of the Bode and Nyquist traces.

4.6.2 Responsive Control Layout

To maintain a clean workspace, the team designed a responsive button architecture. While the Magnitude plot remains constant as the primary reference, a single "View Toggle" button allows the user to cycle the secondary display area between the Step Response, Nyquist, and Phase plots. Similarly, the "Export" button is a multi-function trigger that handles both the structured PDF report and the raw CSV data.

4.6.3 System Initialization and the Welcome FSM

Upon execution, the software initializes through a three-state Finite State Machine (FSM) to ensure a professional "handshake" with the user:

- **State 0 (Fade-in)**: The UI opacity transitions from 0 to 255 over 2 seconds.
- **State 1 (Pulsing)**: A subtle scaling animation provides visual feedback that the system is active and idling.
- **State 2 (Fade-out)**: Triggered by a 3-second timer or a user keypress, the welcome screen clears to reveal the main dashboard.

4.7 The Main Execution Loop

The `main` function serves as the orchestrator of the entire suite. Its execution logic follows a strict sequence to ensure harmony between the UI and the Physics Engine:

1. **Resource Allocation**: Loading of engineering-focused fonts and the SFML rendering window.

2. **Environment Seeding**: Initializing empty input fields with "safe" default values, allowing the user to delete and replace them without causing initialization errors.

3. **The Event Loop**: A high-frequency polling loop that checks for mouse interactions, text entry, and button clicks.

4. **Synchronous Redraw**: If the "Security Guard" detects a valid change in input, the math functions are called in sequence, and the vertex arrays for the plots are updated before the next frame is rendered.